

Summary

The topic synthesizes technical insights regarding the Partition and Quick Sort algorithms, as detailed in the "Algorithm Crash Course for GATE." These algorithms represent a cornerstone of the Divide and Conquer paradigm. The Partition algorithm functions as the primary "divide" step, reorganizing an array around a pivot element such that all smaller or equal elements precede it and all larger elements follow it. Quick Sort leverages this mechanism through recursive calls to sort sub-problems.

Key takeaways include:

- **Partition Logic:** The algorithm utilizes a two-pointer approach (i and j) to reorder elements in $O(n)$ time.
- **Quick Sort Mechanism:** It recursively solves sub-problems defined by the pivot's final position; notably, it does not require an explicit "combine" step.
- **Complexity Variance:** Quick Sort's performance is highly dependent on pivot selection and data distribution, ranging from an optimal $O(n \log n)$ in balanced cases to $O(n^2)$ in worst-case scenarios (e.g., already sorted arrays).
- **Resource Efficiency:** Unlike Merge Sort, which requires $O(n)$ extra space, Quick Sort is often preferred for its in-place sorting capabilities, though specific space complexities vary by implementation.

The Partition Algorithm

The Partition algorithm is the functional engine of Quick Sort. Its objective is to place a chosen pivot element in its correct sorted position within an array while simultaneously segregating other elements based on their value relative to that pivot.

Functional Steps and Logic

The algorithm operates on an array (A) from starting index (P) to ending index (R):

1. **Pivot Selection:** The final element of the array segment is selected as the pivot ($\text{Pivot} = A[R]$).
2. **Pointer Initialization:**
 - A secondary index (i) is initialized to $(P - 1)$. This index tracks the "boundary" of elements known to be less than or equal to the pivot.
 - The primary index (j) iterates from (P) to $(R - 1)$.
3. **Iterative Comparison:**
 - For each element ($A[j]$), the algorithm compares it to the pivot.
 - If $A[j] \leq \text{Pivot}$, the boundary index (i) is incremented, and $A[i]$ is swapped with $A[j]$.

4. **Final Placement:** Once the loop completes, the pivot ($A[R]$) is swapped with the element at $A[i+1]$. This ensures the pivot is situated between the two partitions.
5. **Return Value:** The algorithm returns the final index of the pivot ($i + 1$), which serves as the split point for subsequent recursive calls.

Partition Complexity

The time complexity of the Partition algorithm is $O(n)$. Regardless of whether the pivot is the first, last, or middle element, the algorithm must perform $(n-1)$ comparisons to process the remaining elements in the array segment.

The Quick Sort Algorithm

Quick Sort is a recursive Divide and Conquer algorithm that uses Partitioning to sort an array.

Algorithmic Structure

The algorithm follows a strict logic gated by the condition $(P < R)$ (ensuring at least two elements are present):

- **Divide Step:** The $\text{Partition}(A, P, R)$ function is called, returning an index (q) . This step places the pivot at index (q) .
- **Recursive Solve:**
 - $\text{QuickSort}(A, P, q - 1)$ is called for the left sub-problem.
 - $\text{QuickSort}(A, q + 1, R)$ is called for the right sub-problem.
- **Combine Step:** There is no explicit combine step required because the Partition algorithm places elements in their final positions during the divide phase.

Performance Analysis

Quick Sort's efficiency is determined by how evenly the Partition step splits the array.

Case	Condition	Time Complexity
Best Case	Partitions are balanced (e.g., $k \approx n/2$)	$O(n \log n)$
Worst Case	Partitions are highly unbalanced (e.g., $k = 0$ or $(n - 1)$)	$O(n^2)$
Average Case	Typical distribution of data	$O(n \log n)$

The Worst-Case Scenario

The worst case occurs when the pivot consistently results in a $0 : (n-1)$ split. This frequently happens when the input array is already sorted. In this scenario, the recurrence relation becomes $T(n) = T(n-1) + O(n)$, which resolves to $O(n^2)$ via substitution.

Detailed Execution Trace (Dry Run)

To illustrate the algorithm, consider an array $[5, 7, 3, 1, 10, 2, 4]$ with indices 0 to 6.

Initial Partition (Pivot = 4)

1. Comparisons:
 - $5 > 4$: No action.
 - $7 > 4$: No action.
 - $3 < 4$: (i) becomes 0; swap $(A[0])$ and $(A[2])$. Array: $[3, 7, 5, 1, 10, 2, 4]$.
 - $1 < 4$: (i) becomes 1; swap $(A[1])$ and $(A[3])$. Array: $[3, 1, 5, 7, 10, 2, 4]$.
 - $10 > 4$: No action.
 - $2 < 4$: (i) becomes 2; swap $(A[2])$ and $(A[5])$. Array: $[3, 1, 2, 7, 10, 5, 4]$.
2. Final Swap: Swap $(A[i+1])$ (Index 3) with Pivot (Index 6).
3. Result: Array becomes $[3, 1, 2, 4, 10, 5, 7]$. Partition returns $(q = 3)$.

Subsequent Recursive Calls

The array is now split into two sub-problems based on $(q = 3)$:

1. Left Sub-problem: Indices 0 to 2 $[3, 1, 2]$.
2. Right Sub-problem: Indices 4 to 6 $[10, 5, 7]$.

Following the same logic for the left sub-problem $[3, 1, 2]$ (Pivot = 2):

- $3 > 2$: No action.
- $1 < 2$: (i) moves; swap 1 and 3. Array: $[1, 3, 2]$.
- Final Swap: Swap 3 and 2. Result: $[1, 2, 3]$.

Comparative Context: Quick Sort vs. Merge Sort

While both are Divide and Conquer algorithms, they differ significantly in resource management:

Feature	Merge Sort	Quick Sort (Partitioning)
Space Requirements	Requires an auxiliary array of size $O(n)$, making it memory-intensive for large datasets.	Performed in-place, generally more space-efficient.

Predictability	Fixed division point (middle), ensuring consistent ($O(n \log n)$) performance.	Division point depends on pivot selection, leading to ($O(n^2)$) worst-case risk.
Combine Step	Performs its work in the "Merge" (combine) phase.	Performs its work in the "Partition" (divide) phase.

